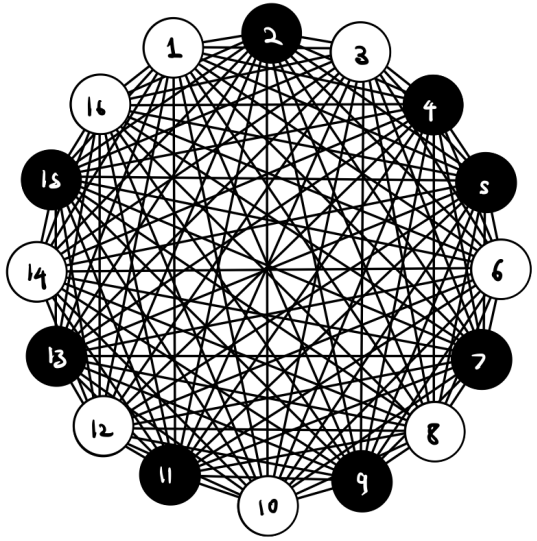
Statistical mechanics and machine learning, the 2024 Nobel prize

Aditya Khanna

Introduction

- Hopfield networks and Boltzmann machines
- Laid the groundwork for AI/ML
- Won the Nobel prize in Physics in 2024

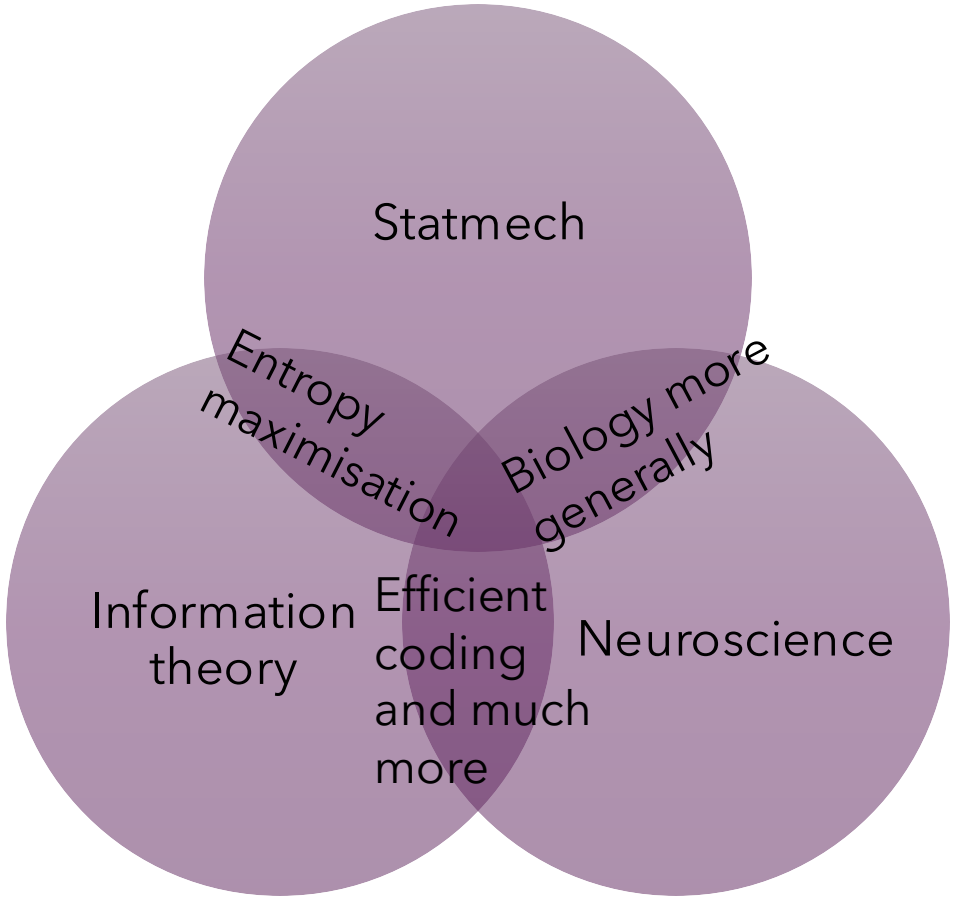






Key point

- Mathematical isomorphisms between results from statistical mechanics, neuroscience and information/probability theory.
- Focus on relationships relevant to ML



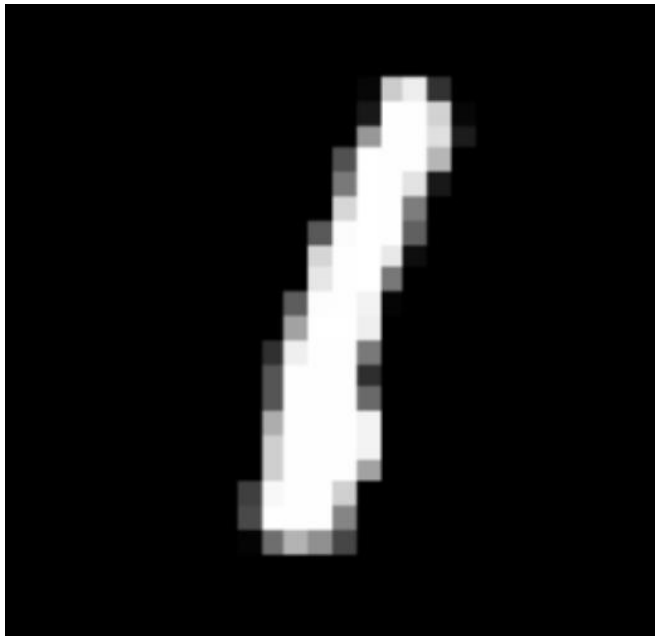


Hopfield networks – a model of associative memory in the brain

- So what is associative memory?

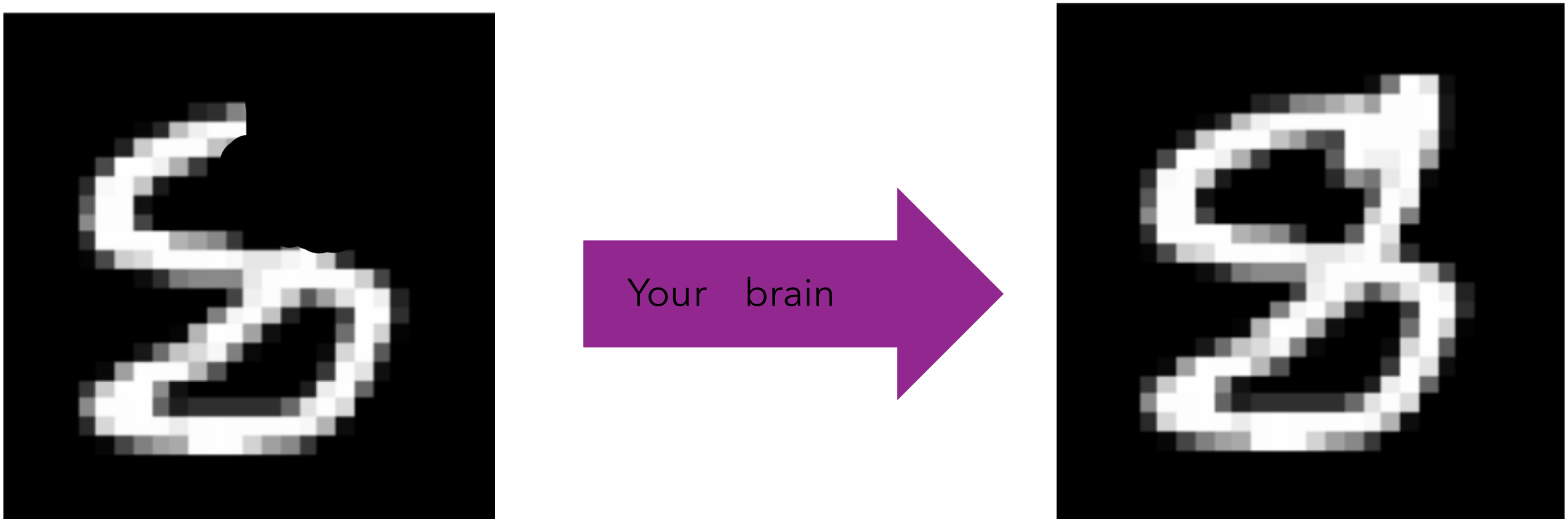
What is associative memory?





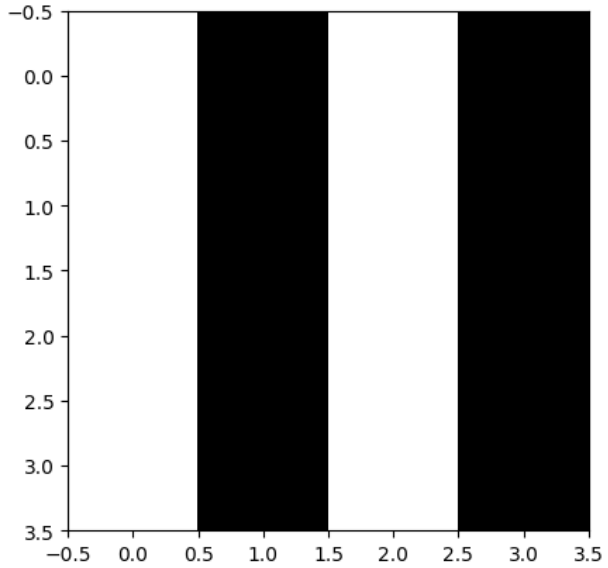


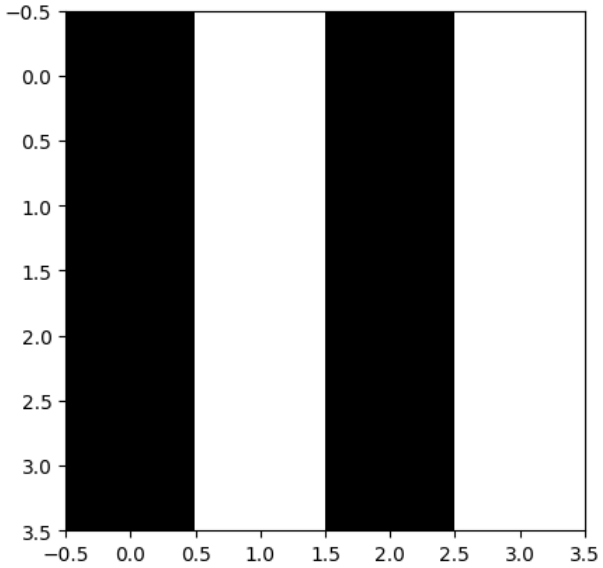
Obscured number from the previous slide



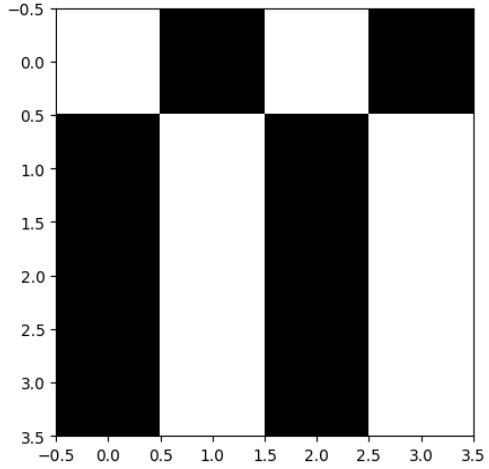
Intro to Hopfield: structure and function

Memories:

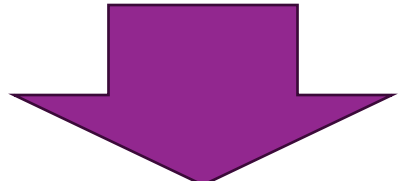




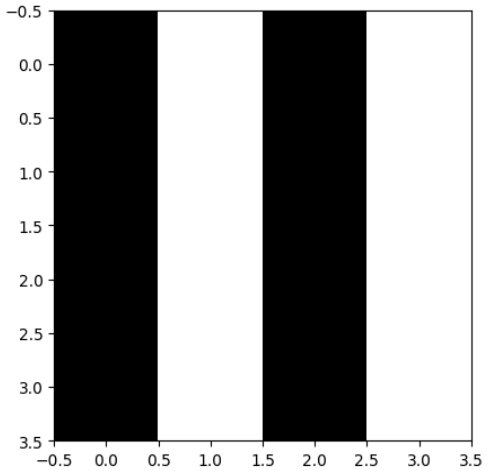
Input:



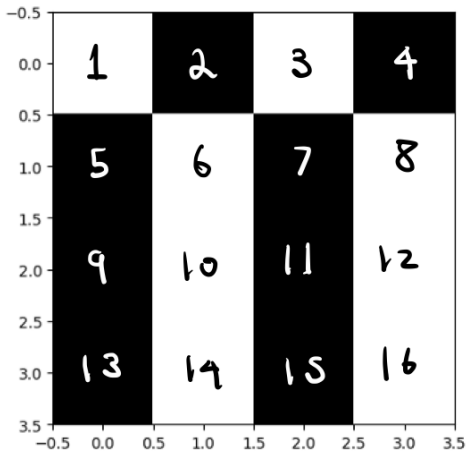
Algorithm:

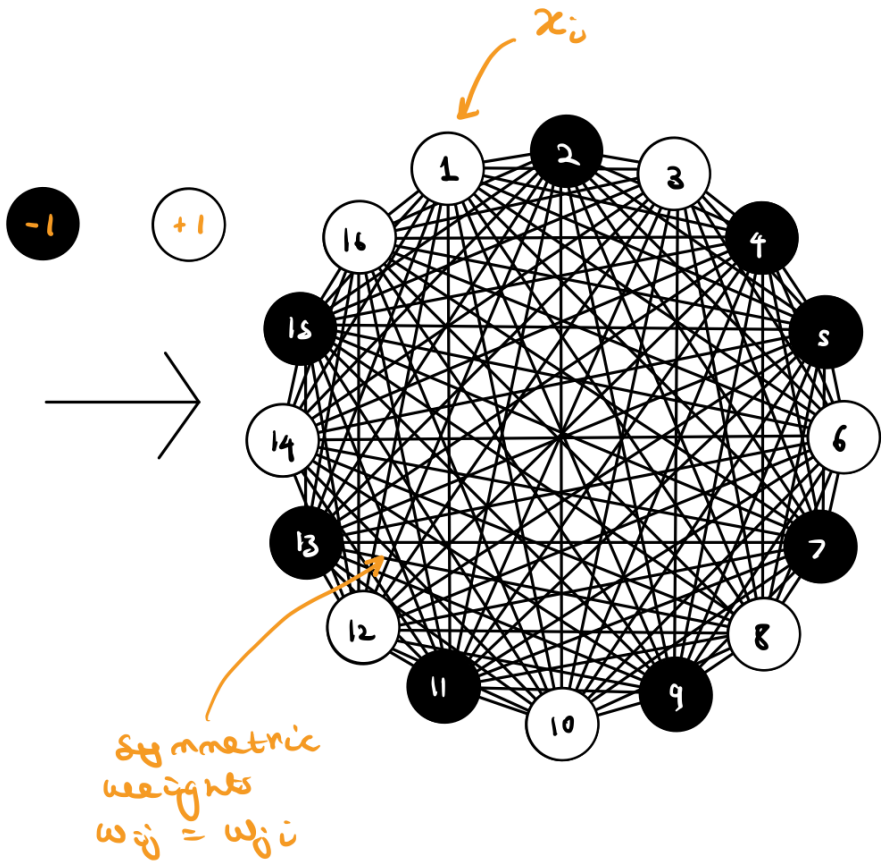


Output:

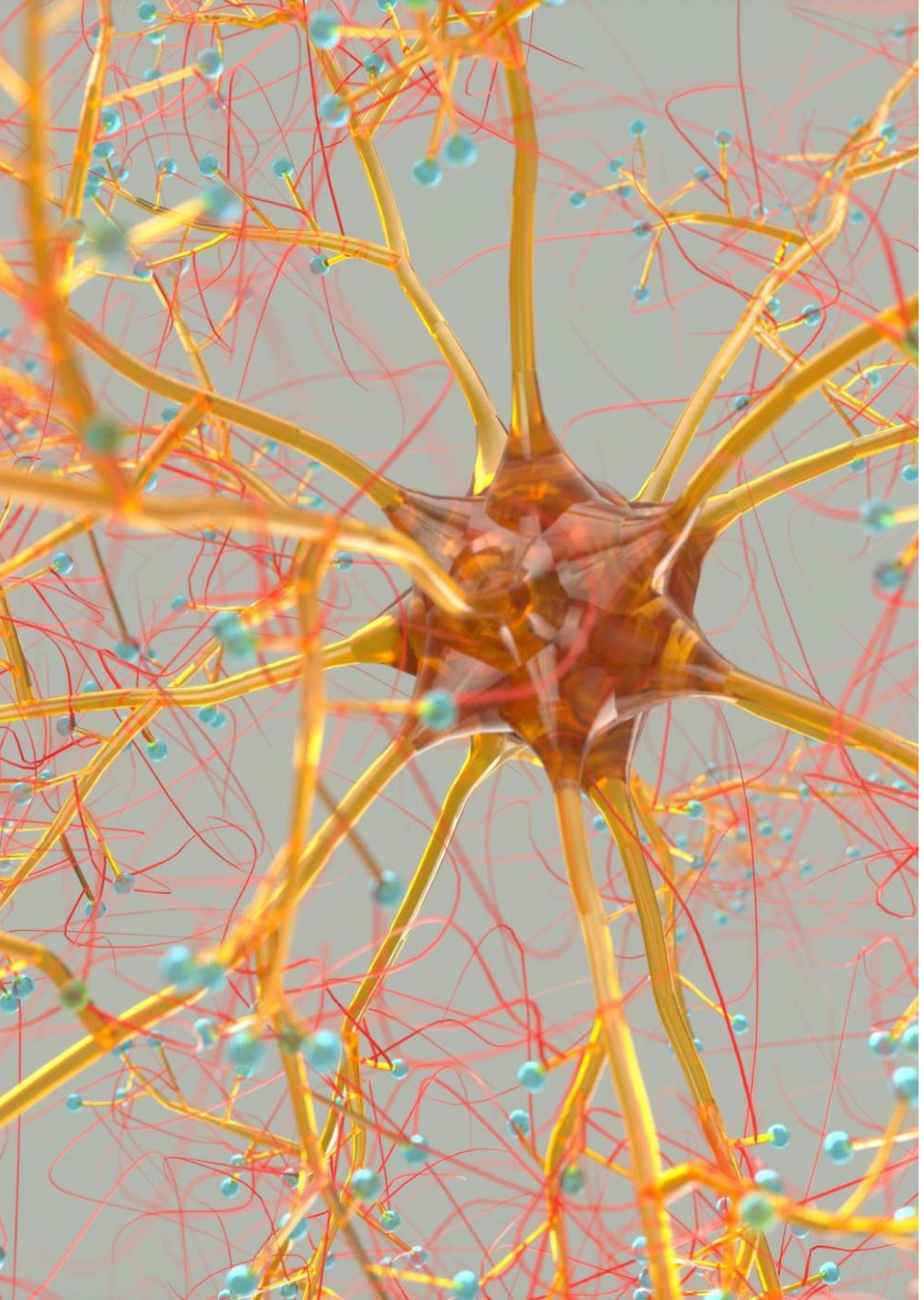


Intro to Hopfield: structure of a Hopfield network





- Consider all of the pixels of the canvas as neurons that can be +1 or -1, and connect them all up with specific symmetric "weights" between them.
- "Learning rule" decides the weights, the effects of neurons upon each other
- "Update rule" takes the network from an initial state to the memory we want



Motivation: Hebbian learning rule

- "neurons that fire together wire together"
- Part of the neuroscientific explanation of associative memory, if neuron A and neuron B fire at the same time at some point in the past, this results in a stronger connection between them.
- So if A fires, then B has a higher chance of firing.
- In the structure we just talked about, this corresponds to a learning rule that **increases** weights between two neurons that tend to agree i.e. are both +1 or both -1.
- This determines our "learning rule"

Learning rule

- Let's mathematically formalise the Hebbian learning rule:
- μ is the index of the pattern, with $x_i^{(\mu)}$ referring to the value of the ith node of the μth pattern.
- So if both neurons are –1 or +1, the weight is increased
- If the neurons are opposite parity i.e. if one neuron fires and the other one doesn't, the weight is decreased

$$\omega_{ij} = 1/p \sum_{\mu=1}^{p} x_i^{(\mu)} x_j^{(\mu)}$$

Motivation: The Ising model

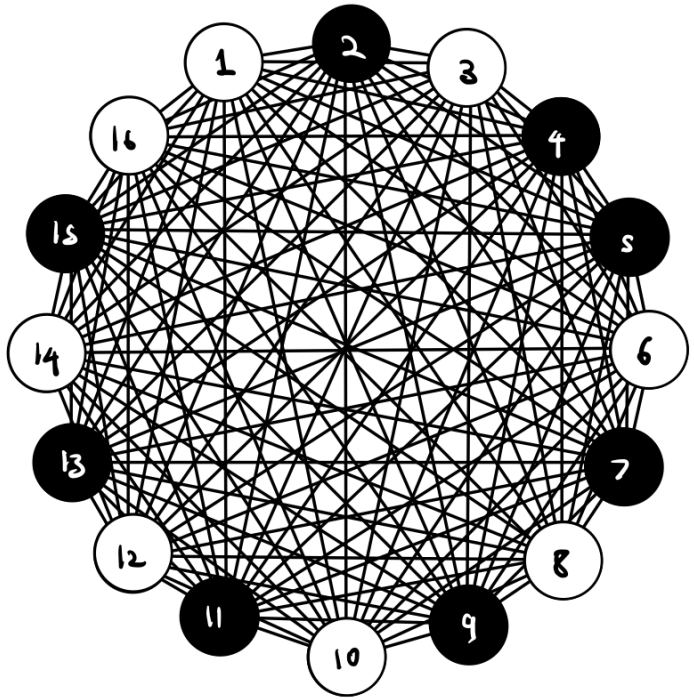
- Simple model from statistical mechanics, describing a lattice of interacting spins in a heat bath e.g. a ferromagnet
- At zero temperature we descend to energy minima
- In our case we want the memories to be energy minima, and we descend to a memory starting in some initial state.
- This determines our "update rule"
- Neighbouring interactions only

$$H = -\sum_{i,j} J_{ij}\sigma_i\sigma_j + h\sum_i \sigma_i$$

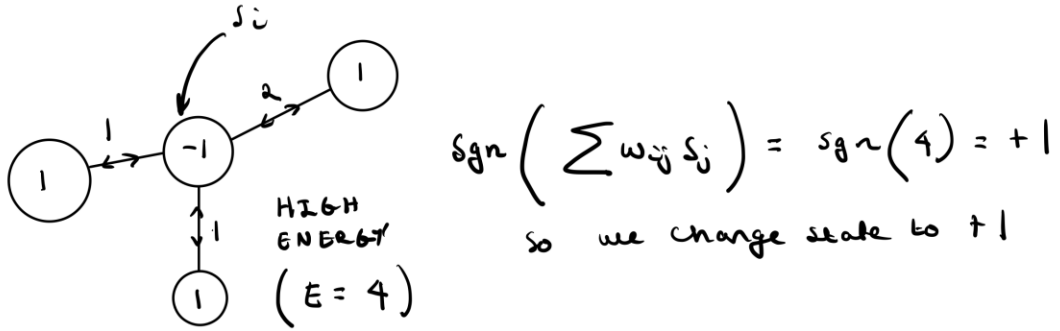
Update rule

- Let's modify the Ising model Hamiltonian to fit our Hopfield network. Now it's not just neighbouring interactions but between all neurons and weighted.
- Intuitively, the energy is lowered when two neurons agree and the weight between them is positive (i.e. they tend to agree in memories) or when two neurons disagree and they tend to disagree in memories.
- This energy is minimum for states corresponding to memories
- The analog of the Ising model comes out naturally from the Hebbian learning rule!

$$E(\vec{x}) = -\frac{1}{2}\sum_{i \neq j} \omega_{ij} x_i x_j = -\frac{1}{2}\vec{x}^T W \vec{x}$$



Update rule cont.



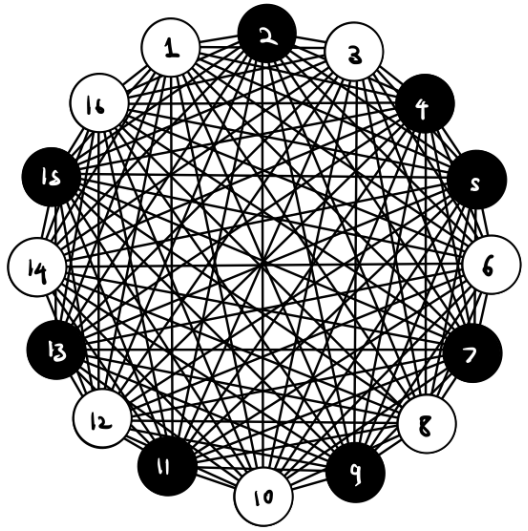
- So now, if we start in an initial state, we want a rule to update the values of neurons until we reach a minimum energy state:
- This update rule for a particular neuron goes through each adjacent neuron and finds the weighted average and finds the sign. The updated state is then that neuron state replaced with the sign.
- This results in a state with an energy that is necessarily less than or the same as it was before.
- We can can sweep and update every neuron asynchronously, or at the same time.
- A stable, energy minimum is reached when none of the neurons change as a result of an update.

$$x_j(t+1) = sgn(\sum_{i \neq j} \omega_{ij} x_j(t))$$

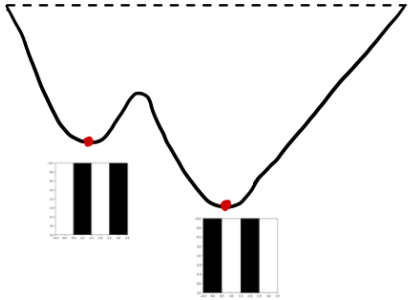
$$\text{sign}(x) = \begin{cases} +1, & \text{if } x > 0 \\ -1, & \text{if } x < 0 \\ 0, & \text{if } x = 0 \end{cases}$$

Summary of Hopfield Networks

The NETWORK:

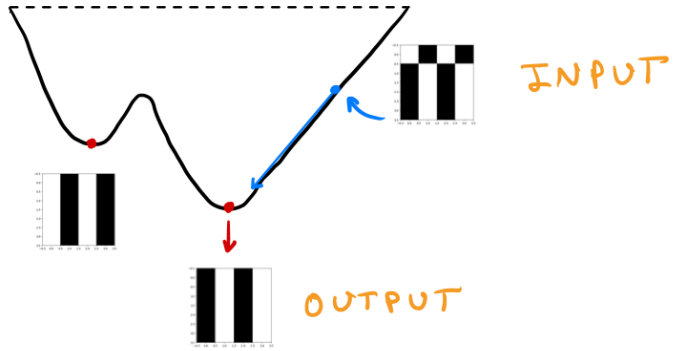


1) Adjust weights (sculpt energy landscape)



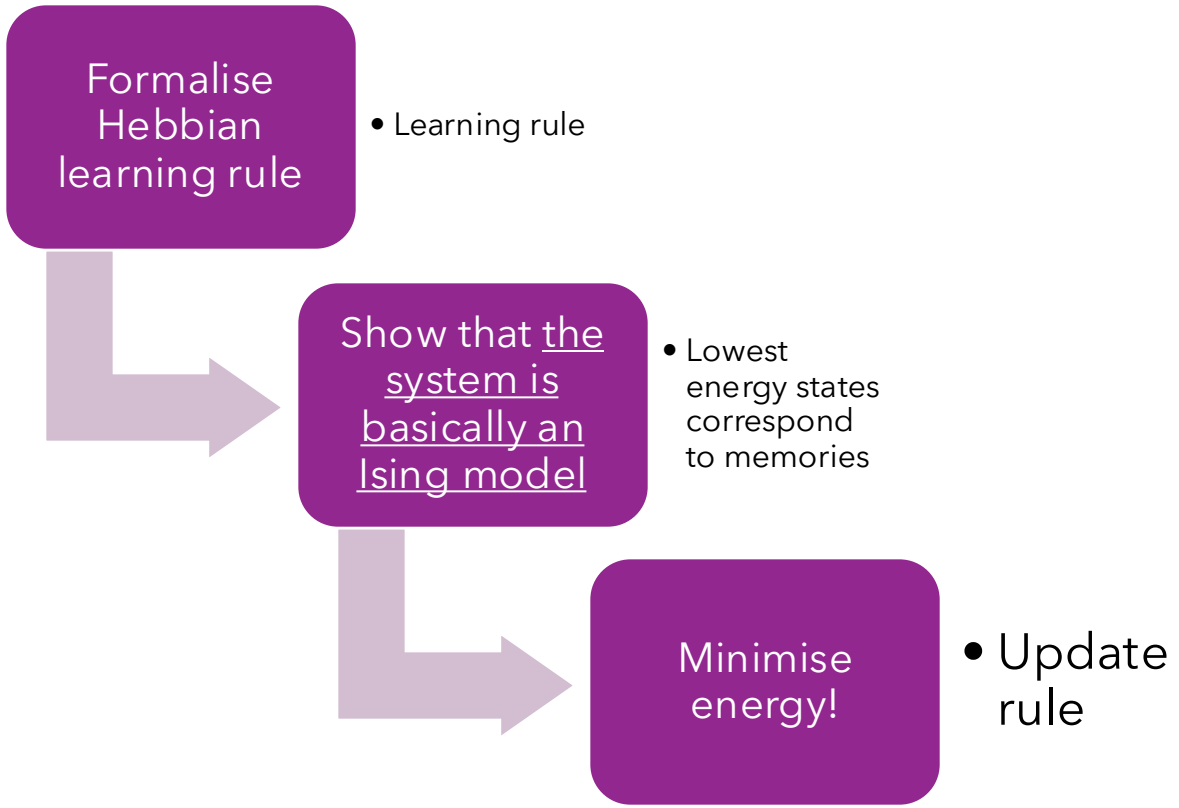
$$\omega_{ij} = 1/p \sum_{\mu=1}^{p} x_i^{(\mu)} x_j^{(\mu)}$$

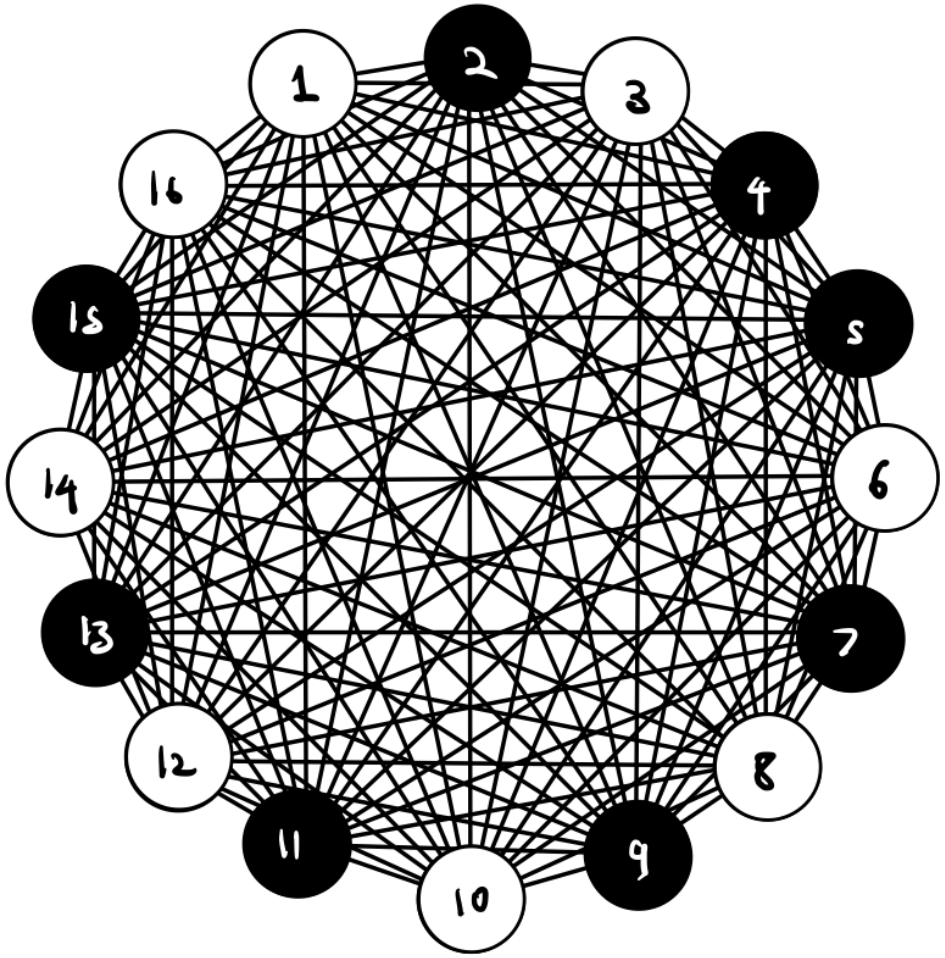
2) Start at input state and descend to energy minima



$$x_j(t+1) = sgn(\sum_{i \neq j} \omega_{ij} x_j(t))$$

Flowchart of motivation





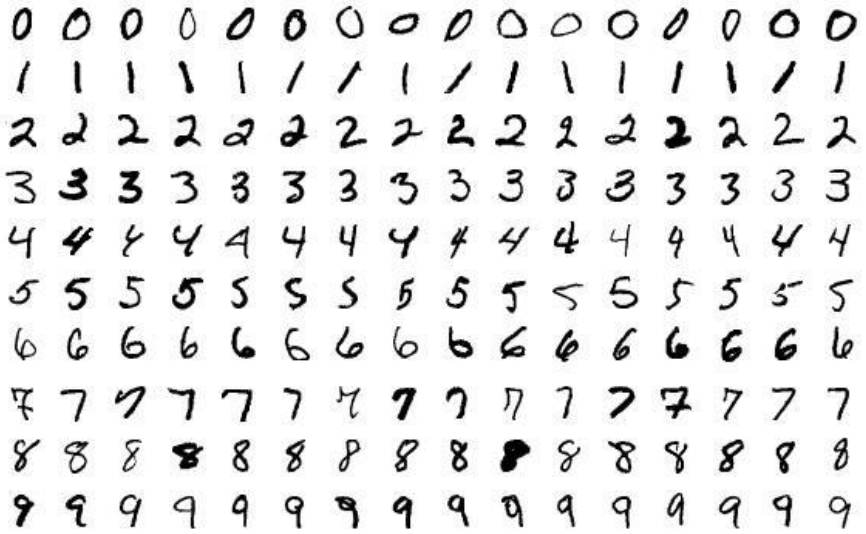
Boltzmann machines - introduction

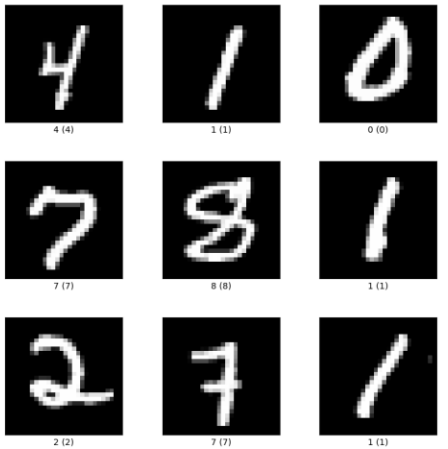
- Boltzmann machines have exactly the same structure and energy function as Hopfield networks, now we'll discuss their uses and how they differ
- We also teach the Boltzmann machine by tuning its weights

$$E(\vec{x}) = -\frac{1}{2}\sum_{i \neq j} \omega_{ij} x_i x_j = -\frac{1}{2}\vec{x}^T W \vec{x}$$

Function of Boltzmann machines

- The idea behind Boltzmann machines is to **generate new samples that mimic the probability distribution of the data**
- Statisticians call this a **generative model**





Boltzmann machines

- Hopfield networks had a **deterministic** update rule, i.e. make a specific change that will necessarily decrease the energy of the system.
- Hinton introduced a **stochastic** update rule, neurons can flip randomly even if doing so increases the energy, depending upon a probability distribution determined by a value called the **temperature.**
- This involves considering an Ising model where we maximise entropy but also maximise the likelihood of observing the data under a particular temperature.

The physics of it

- The Ising model at zero temperature seeks to minimise its Hamiltonian.
- By increasing the temperature, we introduce thermal fluctuations.
- At a non-zero temperature, the probability of being in a microstate follows a **Boltzmann distribution** (comes from maximising entropy under the constant temperature constraint)
- We want to mimic the probability distribution of the data under a particular temperature
- We can use a clever sampling technique to construct a Monte Carlo method that samples from this distribution.

$$Z = \sum_{\mathbf{x}} \exp(-\beta E(\mathbf{x})).$$

$$P(\vec{x}) = \frac{e^{-E(\vec{x})/T}}{Z}$$

$$H = -\sum_{i,j} J_{ij}\sigma_i\sigma_j + h\sum_i \sigma_i$$

From statistical mechanics to machine learning

- We maximise entropies in both Statistical Mechanics and information theory. In I-T we want the *fairest* distribution. In S-M, entropy is maximised at equilibrium.
- The entropy we maximise here is the information theoretic one.
- Over here, “energy” of a configuration refers to its closeness to the training data, and the temperature constant constrains the “mean energy”, getting us the Boltzmann distribution
- The Boltzmann distribution here essentially makes states closer to the data (low energy) more “thermally accessible”

Boltzmann machine learning

- The Hamiltonian remains the same as before, but we now reframe it in terms of a weight matrix Wij, where x is a vector representing the states of the spins
- We want to tune weights so that the probability of observing the data (given by Boltzmann) is maximised, I.e. maximise the probability of observing the data given the weights **at a given temperature**

$$E(\mathbf{x}) = -\frac{1}{2}\mathbf{x}^{\mathsf{T}}\mathbf{W}\mathbf{x}$$

$$P(\mathbf{x}\,|\,\mathbf{W}) = \frac{1}{Z(\mathbf{W})}\exp\left[\frac{1}{2}\mathbf{x}^{\mathsf{T}}\mathbf{W}\mathbf{x}\right]$$

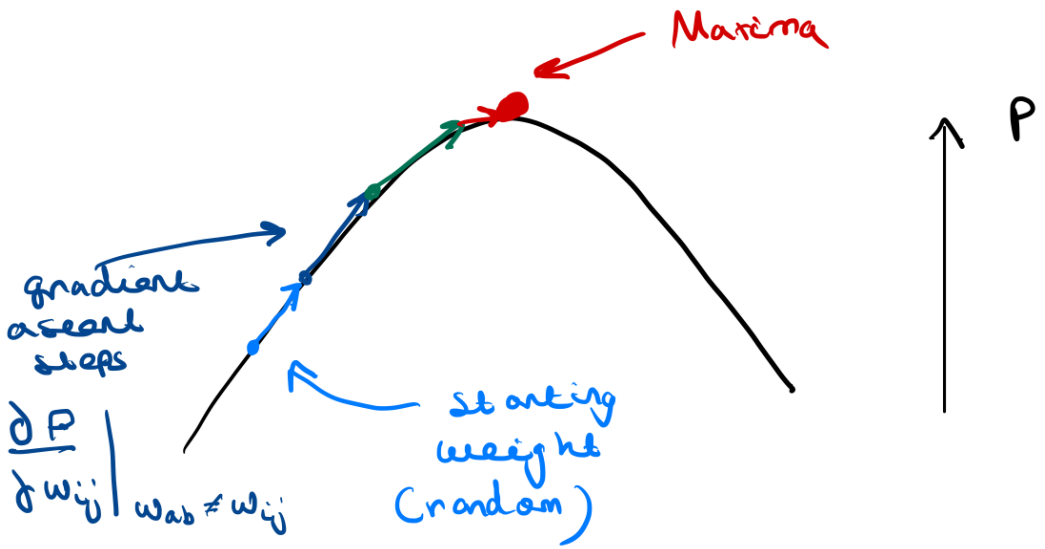
$$\left[\prod_{n=1}^{N} P(\mathbf{x}^{(n)}\,|\,\mathbf{W})\right]$$

Boltzmann machine learning continued

- Let us first try to find weights that maximise the probability of observing the input data (under the free parameter “T”)
- If we have n samples of data, we can take the log likelihood of the probability with respect to the weights!

$$\ln\left[\prod_{n=1}^{N} P(\mathbf{x}^{(n)} \,|\, \mathbf{W})\right] = \sum_{n=1}^{N}\left[\frac{1}{2}\mathbf{x}^{(n)\mathsf{T}}\mathbf{W}\mathbf{x}^{(n)} - \ln Z(\mathbf{W})\right],$$

$$\begin{aligned}\frac{\partial}{\partial w_{ij}} \ln P(\{\mathbf{x}^{(n)}\}_1^N\} \,|\, \mathbf{W}) &= \sum_{n=1}^{N}\left[x_i^{(n)}x_j^{(n)} - \langle x_i x_j\rangle_{P(\mathbf{x}\,|\,\mathbf{W})}\right] \\ &= N\left[\langle x_i x_j\rangle_{\text{Data}} - \langle x_i x_j\rangle_{P(\mathbf{x}\,|\,\mathbf{W})}\right]\end{aligned}$$



Boltzmann machine learning continued

- Now that we have the gradient of the log-likelihood wrt each of the weights, it becomes a simple matter of gradient ascent optimisation to find the weights that maximise probability.
- The gradient points in the direction of steepest ascent, so we numerically find the maximum (I.e. least negative) by taking steps in this direction.
- But how exactly do we interpret this gradient ascent learning rule?

Interpretation of the grad log likelihood

- What is the deeper meaning of this gradient that we are optimising?
- Well it all comes back to the Hebbian learning rule!

$$\frac{\partial}{\partial w_{ij}} \ln P(\{\mathbf{x}^{(n)}\}_1^N \} \,|\, \mathbf{W}) = \sum_{n=1}^{N} \left[x_i^{(n)} x_j^{(n)} - \langle x_i x_j \rangle_{P(\mathbf{x}\,|\,\mathbf{W})} \right]$$

$$= N \left[\langle x_i x_j \rangle_{\text{Data}} - \langle x_i x_j \rangle_{P(\mathbf{x}\,|\,\mathbf{W})} \right]$$

The first term: empirical correlation between two nodes over the course of the data

- The first term in the grad represents the mean correlation over all the data
- Reminder: $x_i^{(n)}$ is the value (1 or -1) of the ith node of the nth memory. So we find this correlation over all memories
- Essentially the Hebbian learning rule!
- The gradient seeks to increase weights between neurons that agree in the data, nothing unexpected.

$$\langle x_i x_j \rangle_{\text{Data}} \equiv \frac{1}{N} \sum_{n=1}^{N} \left[x_i^{(n)} x_j^{(n)} \right],$$

The second term

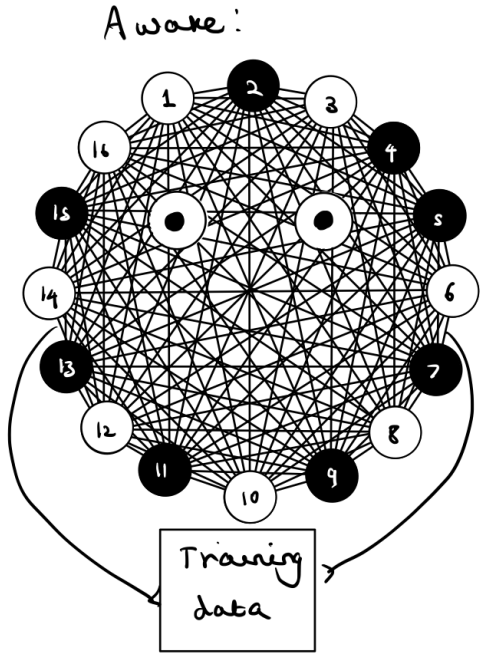
- Now we TAKE AWAY a term
- Instead of the correlation of two nodes/neurons over the course of the data, we find the correlation over the operation of the Boltzmann machine
- This term tends to decrease weights that agree while the model is running!

$$\langle x_i x_j \rangle_{P(\mathbf{x}\,|\,\mathbf{W})} \equiv \sum_{\mathbf{x}} x_i x_j P(\mathbf{x}\,|\,\mathbf{W}).$$

The wakeful/sleepy Boltzmann machine

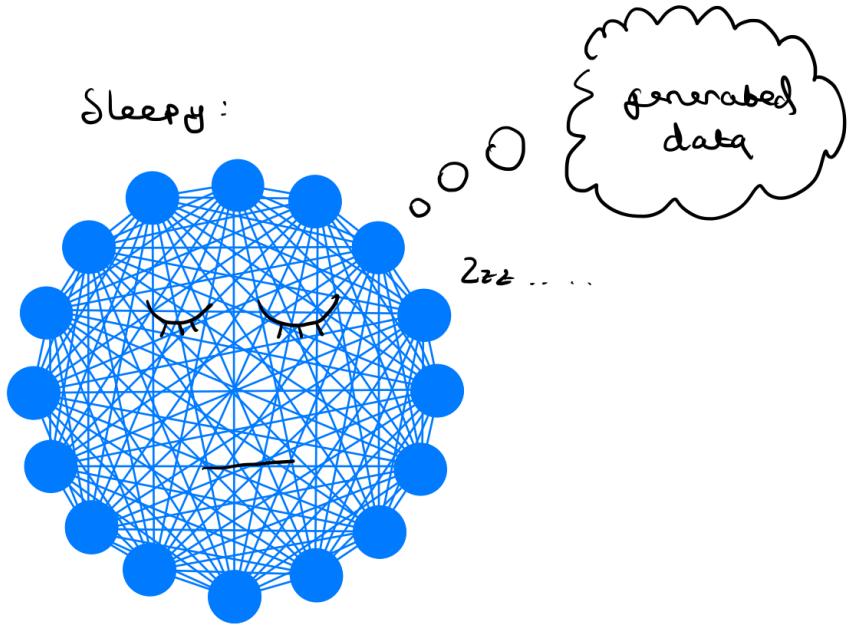
$$\frac{\partial}{\partial w_{ij}} \ln P(\{\mathbf{x}^{(n)}\}_1^N \,|\, \mathbf{W}) = \sum_{n=1}^{N} \left[x_i^{(n)} x_j^{(n)} - \langle x_i x_j \rangle_{P(\mathbf{x}\,|\,\mathbf{W})} \right]$$

- So one way to interpret both of these terms is, the standard Hebbian learning rule corresponds to the Boltzmann machine being awake, measuring correlations in real world data.
- The second term is the Boltzmann network dreaming, measuring correlations in the samples it produces.
- If these correlations are essentially equal, the two terms cancel out and as we'd expect and want, the weights don't change too much.



"learning"

$$\langle x_i x_j \rangle_{\text{Data}} \equiv \frac{1}{N} \sum_{n=1}^{N} \left[x_i^{(n)} x_j^{(n)} \right],$$

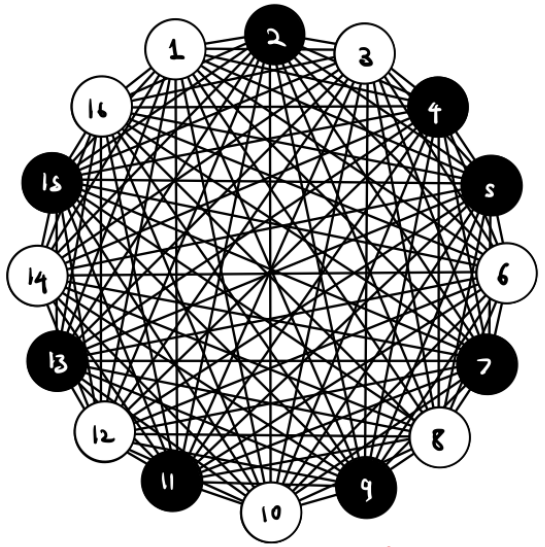


"dreaming"

$$\langle x_i x_j \rangle_{P(\mathbf{x}\,|\,\mathbf{W})} \equiv \sum_{\mathbf{x}} x_i x_j P(\mathbf{x}\,|\,\mathbf{W}).$$

Sampling from a Boltzmann machine

- So now that we've tuned the weights, how do we get it to actually generate new samples that mirror the probability distribution?
- We can initialise our Boltzmann machine completely randomly.
- Construct a transition rule that has a stationary distribution corresponding to the Boltzmann distribution (c.f. Metropolis algorithm)
- Modern techniques use contrastive divergence



RANDOM initialisation

(1) Propose flip (randomly or via a sweep)

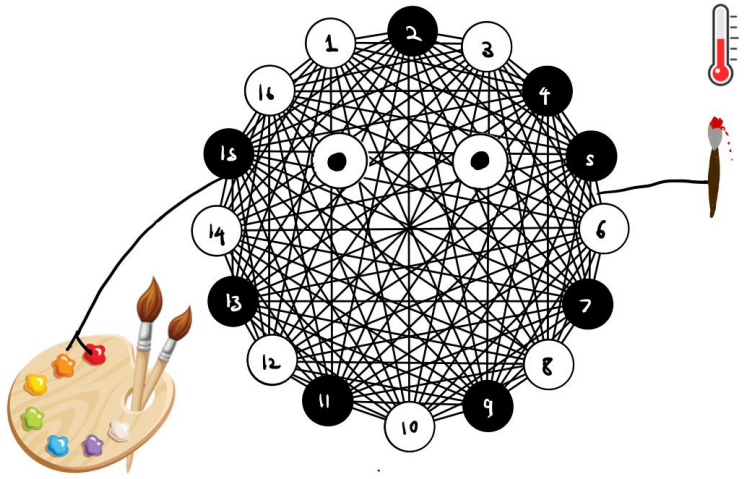
(2) Calculate energy

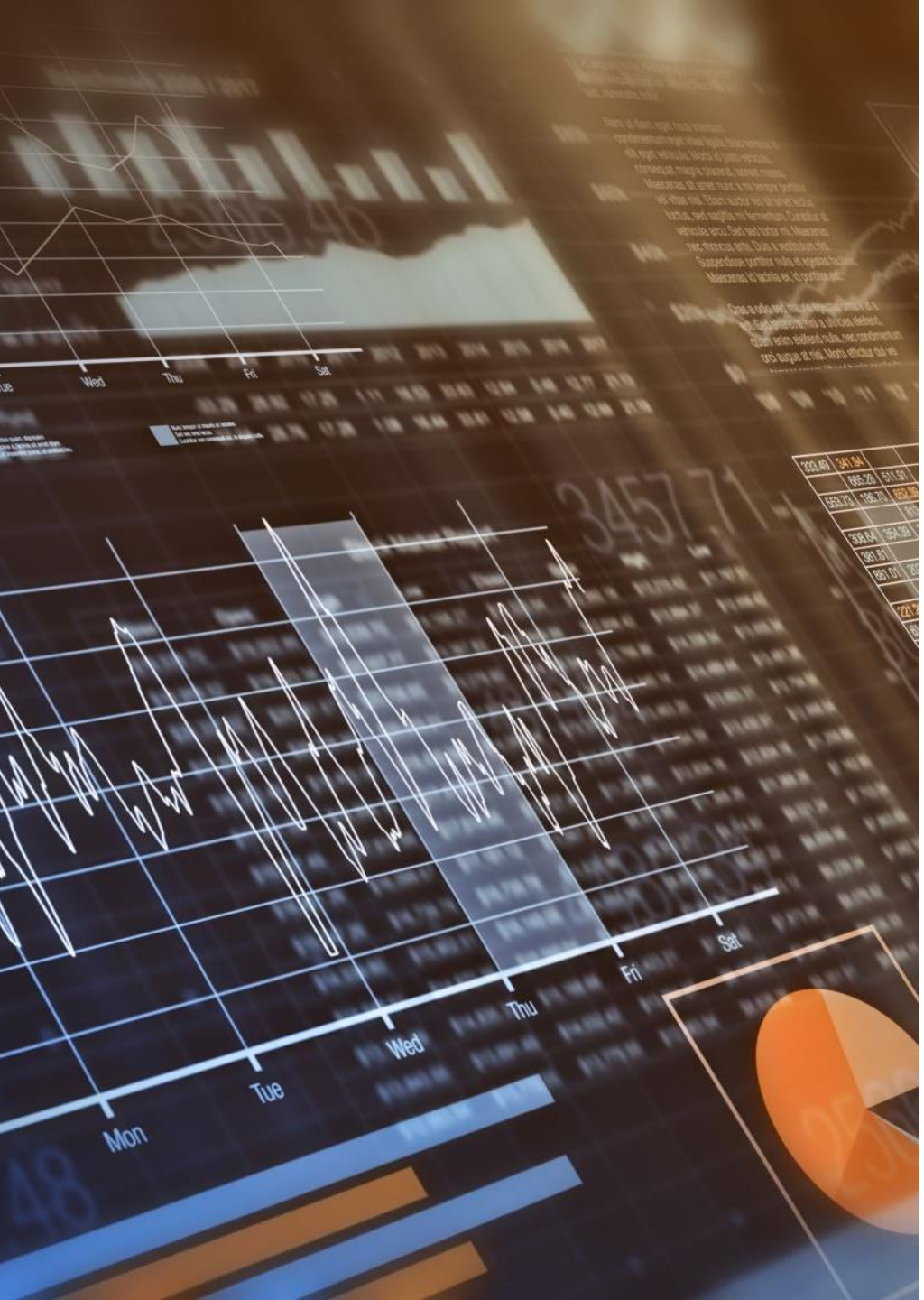
(3) Transition to proposed state with probability $e^{-\frac{\Delta E}{T}}$ if < 1, else transition

This thing called ‘temperature’

- Increasing the temperature makes higher energy states more thermally accessible (and thus probable)
- This makes the Boltzmann machine have a higher probability of producing samples that diverge more from the training data.
- Intuitively this may correspond to “creativity”
- At T=0 we get a Hopfield network

$$P(\vec{x}) = \frac{e^{-E(\vec{x})/T}}{Z}$$



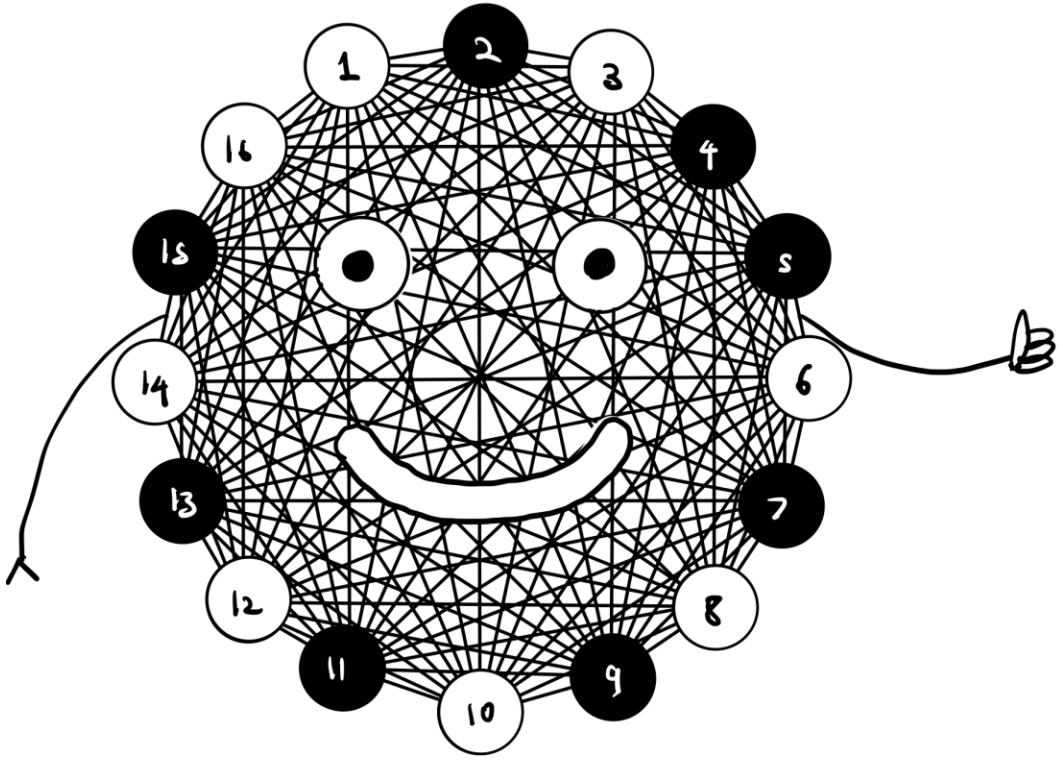


Summary

- We can use our knowledge of statistical mechanics (and neuroscience) to do interesting things in machine learning.

Thank you !

- References and further reading:
- Information theory, inference and learning algorithms – David MacKay 2003
- A Brain-Inspired algorithm for memory – Artem Kirsanov 2024
- Generative model that won the 2024 Nobel Prize – Artem Kirsanov 2024





Uses of Boltzmann machines

- Boltzmann machines essentially learn complex probability distributions, and thus are incredibly widespread and useful. Here are just some examples:
- Recommender systems, like Spotify and Netflix
- Optimisation problems in Quantum Computing and elsewhere.
- Synthetic data generation

So did it deserve the Nobel prize in Physics?

- The nature of this research was highly interdisciplinary
- It therefore raises questions about how massively impactful research that combines multiple disciplines should be fairly rewarded in STEM.

Flowchart of motivation (2)

Start with an Ising model with nonzero temperature

Maximise likelihood AND entropy under constant temperature

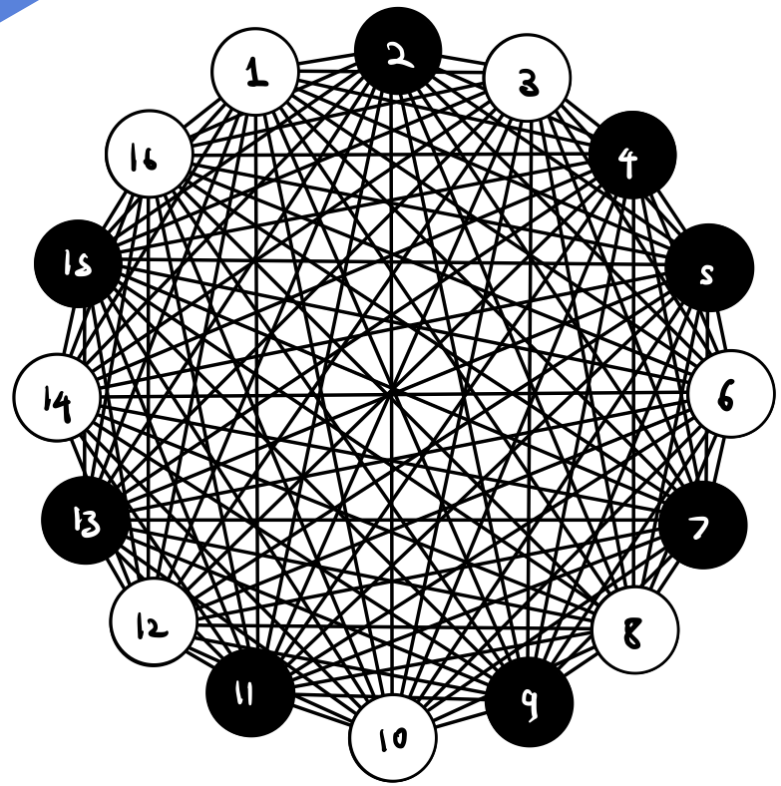
- I.e. Maximum likelihood estimator with Boltzmann distribution at T

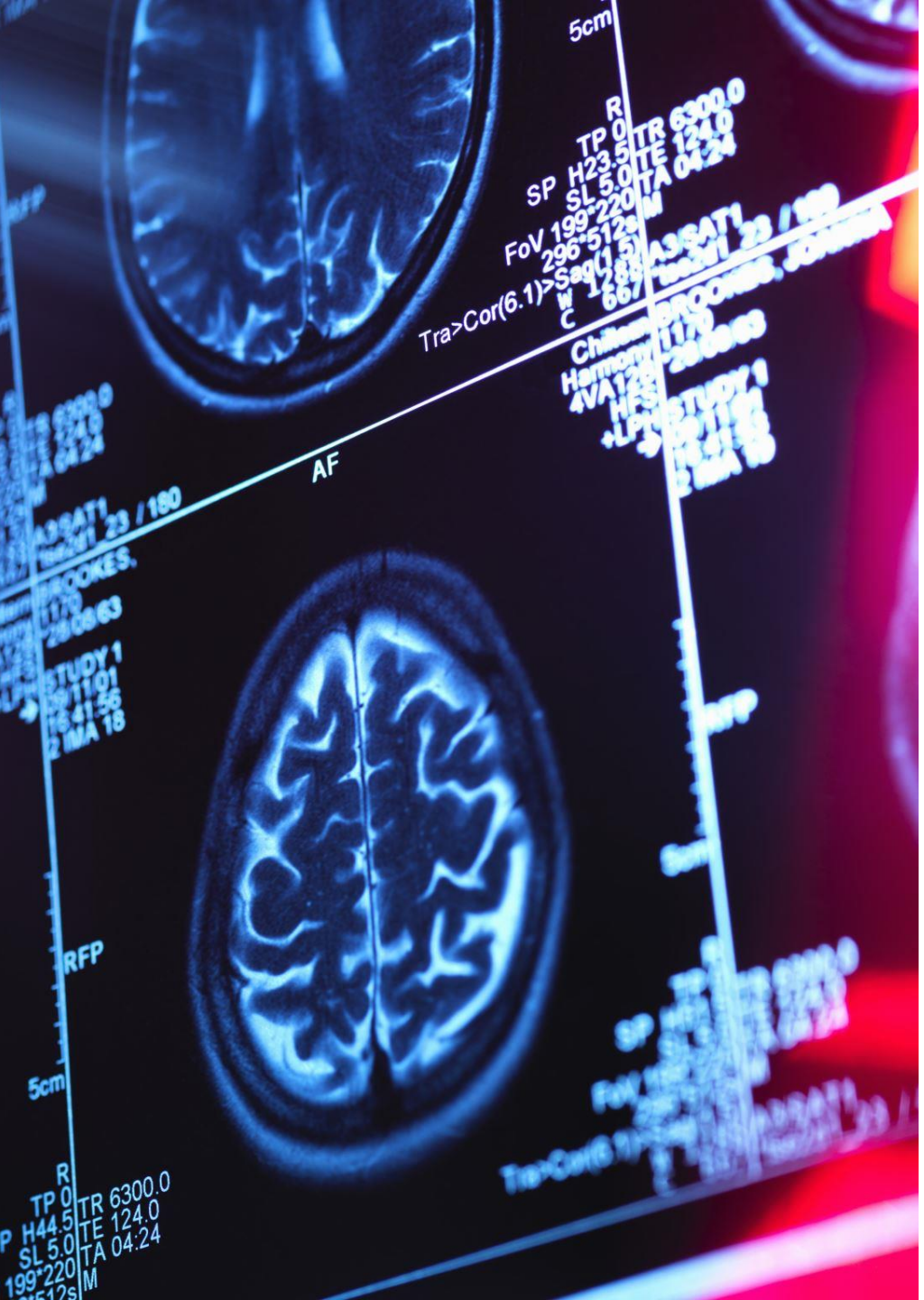
$$P(\mathbf{x} \mid \mathbf{W}) = \frac{1}{Z(\mathbf{W})} \exp\left[\frac{1}{2}\mathbf{x}^{\mathrm{T}}\mathbf{W}\mathbf{x}\right]$$

Get out a modified Hebbian learning rule to learn weights

- Initialise randomly, and sample to produce new ones!

$$\frac{\partial}{\partial w_{ij}} \ln P(\{\mathbf{x}^{(n)}\}_1^N\} \mid \mathbf{W}) = \sum_{n=1}^{N} \left[x_i^{(n)} x_j^{(n)} - \langle x_i x_j \rangle_{P(\mathbf{x} \mid \mathbf{W})}\right]$$





Uses of Hopfield networks

- As a basic computational neuroscience model of associative memory formation
- Combinatorial/graph optimisation problems
- Early pattern recognition systems (in the 1990s)
- “Modern Hopfield Networks” – recent research for LLMs